

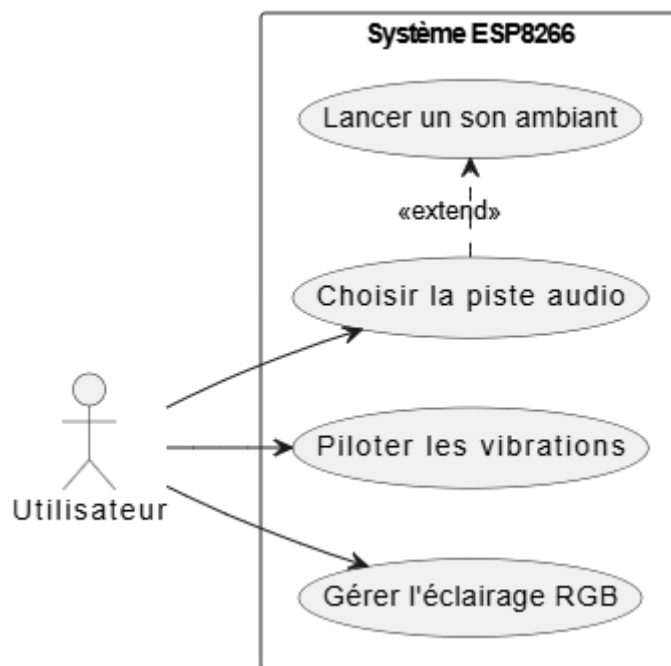
Introduction

Notre projet consiste en la création d'une lampe connectée pour une meilleur immersion dans des jeux de rôles entre amis.

Pour cela nous avons utilisé une ESP8266 qui nous permet de réaliser une interface de commande pour un utilisateur, et ce grâce à une page Web HTML permettant de contrôler les différents capteurs et actionneurs que disposent la lampe.

Cas d'utilisation

Voici le cas d'utilisation de notre lampe connectée :



Scénario

Voici des exemples de quelques scénarios qui détaillent le cas d'utilisation

Scénario 1 : Sélection et lecture d'un son ambiant

- Acteur : Utilisateur.
- Action : L'utilisateur choisit un titre (ex: "Zelda Mechanism") sur la page web.
- Événements : * Le navigateur envoie une requête /sound?play=Zelda+Mechanism.
 - Le WebServerController utilise sa std::map pour trouver l'ID de la piste correspondant au texte.
 - Le DFPlayerMini envoie la commande série au module MP3.
- Résultat : Le haut-parleur diffuse le son sélectionné.

Scénario 2 : Pilotage de l'intensité des vibrations

- Acteur : Utilisateur.
- Action : L'utilisateur déplace le curseur "vibrations" sur son smartphone.
- Événements : * Une requête HTTP /set?niveau=X est transmise en temps réel.
 - Le contrôleur appelle vibrationMotor.setSpeed(X).
- Résultat : Le moteur ajuste sa force de vibration physiquement.

Scénario 3 : Déclenchement manuel de l'animation RGB

- Acteur : Utilisateur.
- Action : L'utilisateur appuie sur le bouton "OFF" pour le passer sur "ON".
- Événements : * Le serveur reçoit l'ordre /RGB_Animation.
 - La méthode matrixController.start(2000) est appelée.
- Résultat : La matrice commence son cycle de changement de couleurs toutes les 2 secondes.

Diagramme de classe

Voici le diagramme de classe de notre lampe connectée :

*Nous avons fait hériter la classe abstraite « Device » à toute les classes et ceux dans le but de mutualiser la fonction d'initialisation « begin ». Et nous avons utilisé la classe abstraite « Actuator » afin d'initialiser la méthode « stop » qui est commune à chaque actionneur.

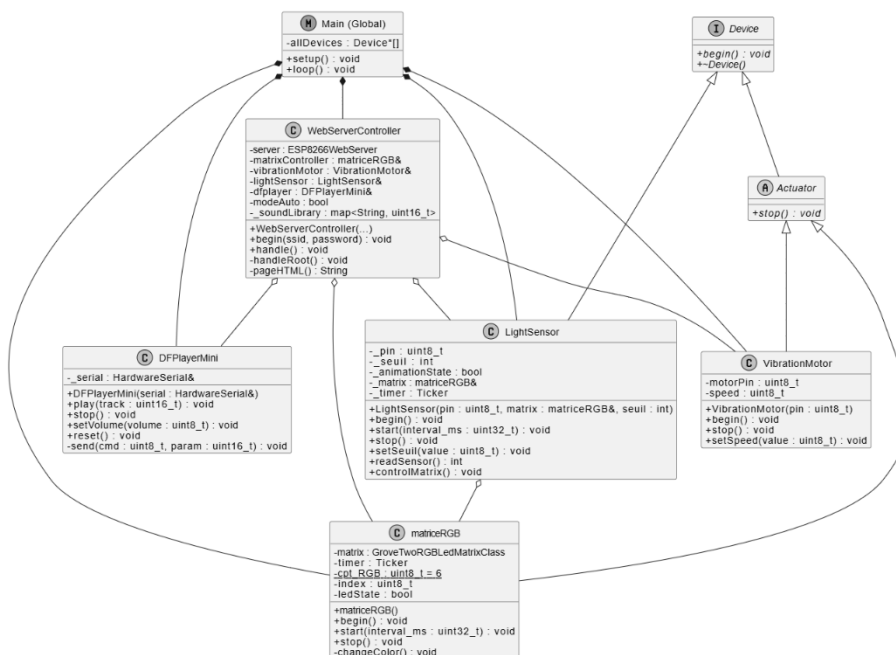
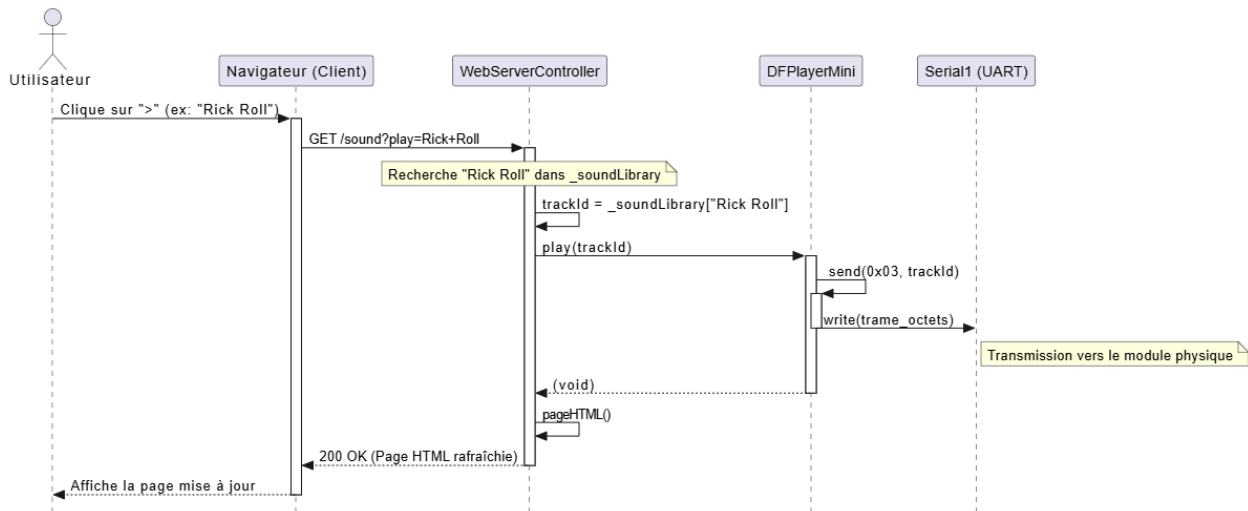


Diagramme de séquence

Voici un diagramme de séquence qui explique comment un utilisateur peut contrôler le son de la lampe depuis l'interface HTML :



Déroulement du projet

Lors de la conception de la lampe, nous avons débuté par le développement de l'interface entre l'ESP8266 et l'utilisateur. Pour ce faire, nous avons créé une page HTML en exploitant la connectivité Wifi native de l'ESP8266. Par la suite, nous avons développé chaque capteur et actionneur de manière indépendante afin de valider leur bon fonctionnement respectif. Enfin, nous les avons intégrés progressivement au système en les reliant à notre interface de commande web.

Fonctionnement du projet

Le fonctionnement du projet est décrit dans le README sur le github :

https://github.com/Melkovich34/BE_POO_Template.git

Conclusion

Notre projet a pu être fini dans les temps et est fonctionnel. Lors de la phase de conception, nous avons eu beaucoup de difficultés à faire fonctionner le lecteur DFPlayer Mini, car l'une des broches du haut-parleur ne fonctionnait plus. Si nous devons améliorer le projet, voici quelques idées que nous pourrions mettre en place :

Gestion des erreurs : Implémenter le mécanisme des exceptions pour gérer les déconnexions WiFi ou les erreurs de lecture I2C de la matrice RGB.

Ajout d'un lecteur RFID : L'idée serait que le maître de jeu lance un chronomètre sur l'application et donne, par exemple, une énigme à un joueur. Si celui-ci réussit à résoudre l'énigme dans le temps imparti, il peut désactiver le chronomètre grâce à la clé RFID et un son de victoire retentit. S'il ne réussit pas, un son de défaite est joué.